

Tree-Search Prototype Feasibility Report — Final

Version: v1 → v2 → v3 Full Arc + Scaling Curve

(Phase F-3)

Generation method: numbers are taken verbatim from `tree_search/harness.py` (v1), `tree_search/harness_v2.py` (v2), `tree_search/harness_v3.py` (v3), **15** tree-state files (`competitions/playground-series-{s3e9,s3e14,s3e5,s3e3,s3e7,s3e1,s3e19,s3e11,s3e16,s3e20}/experiments_tree*.json` , covering the four filename variants `experiments_tree.json` / `experiments_tree_v2.json` / `experiments_tree_v3.json` / `experiments_tree_scale.json`), `docs/scaling_experiment.md` (Phase E-5), and the tree-search appendix of the corresponding `STATUS.md` ; the extraction script is `docs/scripts/build_tree_facts.py` → `docs/tree_facts.json` .

Version scope: this report covers 3 harness generations (v1/v2/v3) + 1 scaling-extension experiment, for a total of **15 tree-search runs covering 10 competitions** (the task brief originally estimated 12 runs; after a precise inventory via `find competitions -iname "experiments_tree*.json"` , the count is 15 — 3 v1 + 9 v2 + 1 scale + 2 v3, see the full results master table below).

Plan context: ERA (Aygün et al. 2026, Nature) — replacing linear single-path iteration with candidate tree search, whose core mechanisms are node scoring, selection rules, plateau detection and backtrack, and "idea injection" that continually expands the candidate set. This report is the final consolidation of the advance validation for Stage 4 (Weeks 6–7) of the plan: v1 (Phase C-2/C-3, 3 competitions) first verifies that the mechanism holds; v2 (Phase D-2..D-6 + E-1..E-4, 9 competitions) adds four upgrades including the ensemble-default node space and sweeps all remaining competitions; E-5 (1 competition) measures the return curve of node budget; v3 (Phase F-1/F-2, 2 validation runs) converges the six lessons discovered during the sweeps into the harness's default behavior. The question is upgraded from "is it worth doing" (v1) to "under what conditions, and with how many evaluations, can it win" (v2), and then to "can the winning mechanism be systematized, and does it hold up under automated real-world validation" (v3).

Contents

1. Purpose and Design	2
Design Goals	2
Correspondence to ERA, and Simplifications	3
2. Full Results Master Table (15 runs, 10 competitions, 3 harness generations)	4
3. v1: Three Competition Results and Per-Competition Analysis (Phase C-2a/b/c)	6
s3e9 (single-model space trapped at the noise ceiling)	6
s3e14 (win: blend node <1s, fold-exact validation)	6
s3e5 (tie: a mature blend space was merely rediscovered; discrete-metric staircase surface)	6
4. When Tree Search Wins — Boundary Conditions Inferred from v1's Three Competitions	7
5. Known Gaps (v1; four fixed/implemented in v2, see next section)	7

6. v2: Four Upgrades and the D Sweep (5 competitions) + E Sweep (4 competitions) — 9 Competitions All Won/Overtuned	8
Four Upgrades (tree_search/harness_v2.py)	8
D Sweep (Phase D-2..D-6, 5 competitions, first comprehensive sweep): 5/5 all wins	8
E Sweep (Phase E-1..E-4, 4 competitions, completing the remaining competitions + two revenge matches)	8
E Sweep Wrap-up	9
7. Priors vs. Local Insight	9
8. Honest Caveats (v1+v2, 9 competitions)	10
9. v3: Six Rules and the F-2 Validation Runs	10
Six Rules (tree_search/harness_v3.py, converging the manual findings from the v1→v2 sweeps into the harness's default behavior)	10
F-2 Validation Runs (s3e7, s3e14, rerun end-to-end with v3's default automatic policy)	11
10. The Full Ten-Competition Coverage Picture	12
11. Scaling-Curve Chapter (Phase E-5, docs/scaling_experiment.md)	13
12. Honest Caveats (v1+v2+v3, all 15 runs)	14
13. Stage 4 Formal Recommendations	15
v3 as the Default Loop	15
Three Engineering Requirements That Must Be Completed Before Deployment (gaps directly identified by the F-2 honest caveats, not a forged new wish list)	15
Budget Rules (validated in Phase E-5, encoded as harness_v3.py constants, Section 11)	15
14. Reproduction Commands (v1 + v2 + v3 + scaling experiment, 15 runs)	16

1. Purpose and Design

Design Goals

`tree_search/harness.py` is a competition-agnostic general engine; each competition need only supply an `evaluate(config) -> score` function. The design choices are as follows:

- **node = one complete, already-scored (or failed) solution:** `{id, parent_id, mutation, config, score, status, wall_s}`. Not an abstract node representing a "partial idea" or "a single hyperparameter change," but a complete, independently reproducible pipeline configuration with a CV score — this is a deliberate simplification, traded for the simplicity of "being able to compare any two nodes at any time."
- **selection rule:** at any moment there is exactly one "active lineage" (the candidate subtree represented by some first-generation child of the root). `select_next_parent()` always picks, within the active lineage, the node with "the best score that has not yet reached `MAX_CHILDREN_PER_NODE` (=3) children" to expand.
- **plateau/backtrack:** whenever the active lineage adds a child, if its score fails to refresh "the global best score before the addition," that lineage's no-improvement counter is incremented by 1; upon reaching `PLATEAU_STREAK` (=3) consecutive times, that lineage is marked "plateaued" and removed

from the candidate set, and `select_next_parent()` switches to the next-best, not-yet-plateaued lineage (i.e., backtrack). Two additional rules prevent the search from getting stuck: (1) a lineage that exhausts its expansion budget is also treated as plateaued; (2) if all lineages plateau simultaneously, clear the plateau flags once (reopen).

- **solo + blend dual node space (from Phase C-2b; upgraded to default from v2):** `solo` (train one model, cache OOF/test predictions to disk) and `blend` (run a weight search over the cached members' OOF, without retraining). This is the key design that fixes the s3e9 lesson: a single-model node space can never reach the score achievable by an ensemble solution.
- **OOF cache:** a solo node's OOF/test matrices are stored as `tree_search/cache_<comp>/solo_<node_id>.npz` , and subsequent blend nodes read them directly without retraining — this is the sole reason blend nodes can achieve "near-free scoring" (this assumption itself was refuted by s3e5 in Section 3 of v1, see below).

Correspondence to ERA, and Simplifications

ERA (Aygün et al. 2026) concept	This prototype's correspondence	What was simplified
Candidate tree replaces single-path iteration	node/lineage/select_next_parent mechanism, fully implemented	none
Plateau detection and backtrack	PLATEAU_STREAK (=3) + lineage exclusion + reopen-once fallback; from v2, metric-aware adaptivity (ADAPTIVE_PLATEAU_STREAK =5)	v1 uses a fixed threshold; only v2 dynamically adjusts by the metric's dispersion
Idea injection	v1: a fixed queue hand-written in the driver script; from v2, <code>suggest_priors</code> does keyword matching against the experience library (a simplified version)	still not "runtime dynamic generation of new directions," only structuring existing experience into a queryable list
Multi-machine parallel search	single-machine sequential ; the <code>harness.py</code> docstring explicitly states "no concurrent <code>add_node</code> calls"	no parallelization
Mutation proposer can self-evolve	fixed to the rule-based queue the agent manually designs at the time of writing the driver script	no runtime idea generation

In short: this prototype fully validates ERA's **search mechanism**, but neither v1 nor v2 yet touches the differentiating capability for which ERA is truly cited — **idea injection**. v3's six rules (Section 9) converge the patterns manually discovered during the v1→v2 sweeps (boundary pushing, late-stage breakthroughs requiring a reopened blend lineage, exploratory bursts) into the harness's own default behavior — in a sense, "distilling the candidate-generation intuition that a human learned across 10 sweep competitions into part of the system" — still not the runtime dynamic generation ERA describes, but closer than v2's static experience-library matching.

2. Full Results Master Table (15 runs, 10 competitions, 3 harness generations)

#	comp	harness ver/Phase	reference best (linear or prior tree)	tree best	verdict	nodes	key
1	s3e9	v1 / C-2a	linear 12.07003	12.07459	loss	20	(non-sing sing spa stru can the ceili
2	s3e14	v1 / C-2b	linear 340.59891	340.52635	win	20	blen type men (div men
3	s3e5	v1 / C-2c	linear 0.56769	0.56766	tie (Δ 0.00003)	40	redi the opti regi gain
4	s3e3	v2 / D-2	linear 0.838140	0.841442	win	22	FEA tenu (ED hyp con
5	s3e7	v2 / D-3	linear 0.899893	0.900242	win	22	prio ense med ded fix
6	s3e1	v2 / D-4	linear 0.557088	0.556329	win	23 (1 failed)	top- awa writ met
7	s3e19	v2 / D-5	linear 10.01946	9.75707	win	22	auto glob (fixe syst low OOF
8	s3e11	v2 / D-6	linear 0.295648	0.295280	win	24	

#	comp	harness ver/Phase	reference best (linear or prior tree)	tree best	verdict	nodes	key
							Cat bou 10—
9	s3e9	v2 / E-1 (revenge match)	v1 tree 12.07459 + linear 12.070034 (full precision)	12.070034	v1 tree win (+0.004556) / linear exact tie	26	ens defa spa hist repr coo refin
10	s3e5	v2 / E-2 (revenge match)	v1 tree 0.56766 + linear 0.56769	0.57066	double win (+0.00300 / +0.00297)	23 (1 failed)	bou pus men (LGR suff weig bud
11	s3e16	v2 / E-3 (acid test)	linear 1.33812 (the only one with a real Kaggle LB anchor)	1.33563	win (−0.186% relative)	27 (3 failed)	raw, inve evid FEA k=8 refin
12	s3e20	v2 / E-4 (structure-dominated)	Phase-B baseline 21.1487	21.0332 (blend, low confidence) / 21.0589 (pure structure)	win	38	JOI axis + YEA new
13	s3e3	v2-scale / E-5 (scaling curve)	v2 tree 0.841442 + linear 0.838140	0.845051	double win	80 (95, incl. 15 dead-end placeholders)	expl KITC (dir full-
14	s3e7	v3 / F-2 (validation run)	v2 tree 0.900242 + linear 0.899893	0.900455	double win	60 (63)	expl meg (pro spa
15	s3e14	v3 / F-2 (validation run)	v1 tree 340.52635 + linear 340.59891	340.35572	double win	60 (62)	expl meg men reta sub weig

Of the 15/15 runs, 12 clearly beat their reference, 2 tied exactly (#9 vs. linear, #3 vs. linear), and 1 clearly lost (#1, whose structural node-space problem was overturned in #9 with v2). Per-version and per-competition analysis is in Sections 3–9; the consolidated verdict over the full ten-competition coverage (taking only each competition's v2/v3 best result against linear, see Section 10) is **9 wins, 1 exact tie, 0 losses**.

3. v1: Three Competition Results and Per-Competition Analysis (Phase C-2a/b/c)

s3e9 (single-model space trapped at the noise ceiling)

Root is an existing regularized single LGB (RMSE 12.11061, 5.2s); all 5 first-generation lineages are **single-model** mutations. All 20 nodes evaluated successfully, total wall-clock 125.4s. The whole-tree best score is the CatBoost seed node itself (12.07459), and none of the 4 backtracks found a better direction.

Lesson: linear iteration's true best score (12.07003) came from a 7-way seed-bagged blend, and the tree search — because its node space was limited to single models — structurally could not reach there. This is not a failure of the search mechanism, but a failure of candidate-space design — it directly prompted s3e14 to add the blend node type, and is also the original motivation for v2 making the ensemble-default node space its first upgrade (Section 9's E-1 shows this structural gap can be overturned with v2 alone).

s3e14 (win: blend node <1s, fold-exact validation)

Root reuses an existing tuned LGB (342.02154, 52.1s). 7 first-generation lineages: 6 solo + 1 BLEND. 20 nodes, 0 failures, 12 solo + 8 blend. The BLEND seed node was the whole-tree best on its first scoring; the **9th** evaluation (node #8, 340.59485) **overtook** the 340.59891 that linear iteration took 7 experiments to find, and finally pulled ahead at node #11 (340.52635). Both backtracks occurred after the best score had already been found.

Reason for the win: the winning node's (#11) 6th member — an LGB that further cut 3 temperature columns from the 21-feature set — was a member combination linear iteration never tried; its value is purely blend diversity, reachable only when the node space can expand to an ensemble.

s3e5 (tie: a mature blend space was merely rediscovered; discrete-metric staircase surface)

Root is an LGB tuned by Optuna directly against post-rounder QWK (0.56244, 1.4s). 8 first-generation lineages. 40 nodes, 0 failures. The tree best is node #11 (a 4-way blend), QWK **0.56766** — differing from the linear best 0.56769 by only **0.00003**, a magnitude so small that the cut-point assignment of a single sample could flip it.

Three new phenomena from the discrete metric: (1) the score surface is staircase-shaped, with ties extremely common (4 groups had identical 5-decimal scores); (2) `PLATEAU_STREAK=3` is effectively stricter for discrete metrics (8/8 lineages all plateaued, the most thorough of the three competitions); (3) the "blend is cheap" assumption does not hold (every candidate weight requires a Nelder-Mead fit, raising a single blend node's cost to ~45–46s, the same order as a solo node) — these three findings directly gave rise to v2's adaptive plateau mechanism.

4. When Tree Search Wins — Boundary Conditions Inferred from v1's Three Competitions

1. **Condition for winning: a blend combination space exists and has not yet been fully exploited by linear iteration** — s3e14 is v1's only win.
2. **Condition for tie/loss: linear iteration has already squeezed out the metric's usable signal, or the node-space design itself is wrong** — s3e5 is the former, s3e9 the latter.
3. **Discrete metrics require adjusting the plateau rule** — metrics like QWK-after-rounder have a staircase-shaped score surface, and a fixed `PLATEAU_STREAK=3` declares each lineage stalled faster than on a continuous metric.
4. **"Blend is cheap" is metric-dependent, not a universal truth** — any search-budget planning premised on "blend nodes being near-free" must first confirm the metric's post-processing cost structure (v3's 6th cost guardrail exists precisely for this).

5. Known Gaps (v1; four fixed/implemented in v2, see next section)

- **Duplicate children:** blend fallback, when the parent config is unchanged, repeatedly produces the same member set for the same parent, wasting plateau quota. → **v2 fixes this with a built-in rejection in `find_duplicate_config / add_node`.**
- **blend-rounder cost:** under a discrete metric (QWK), a blend node's cost catches up to or even exceeds some solo nodes. → **the v2 sweep confirms a further generalization: blend cost varies not only with metric discreteness but also with row count (see Section 6).**
- **No idea injection (ERA's second pillar):** every lineage's mutation queue across the three competitions is a fixed list hand-written in the driver script. → **v2 adds `suggest_priors` (experience-library keyword matching, no LLM call) as a simplified idea injection — still not runtime dynamic generation of new directions, see the honest reading in Section 7.**
- **One tree per competition, no cross-competition transfer:** the three trees are independent of each other. → **v2's `suggest_priors` partially addresses this, but the candidate set itself is still hand-written per competition.**

6. v2: Four Upgrades and the D Sweep (5 competitions) + E Sweep (4 competitions) — 9 Competitions All Won/Overtaken

Four Upgrades (`tree_search/harness_v2.py`)

1. **Ensemble-default node space:** `kind ( solo / blend )` is upgraded to a first-class field of the node. `eval_blend(cache_dir, members, metric_fn, weight_search=...)` runs a Dirichlet or grid-simplex weight search over cached members. Any post-processing rule must be written inside `metric_fn`.
2. **Metric-aware adaptive plateau:** `tie_rate(tree)` measures the fraction of "exactly equal-scoring" nodes among all currently scored nodes; when `tie_rate > TIE_RATE_THRESHOLD` (0.15), the stall threshold is relaxed from `PLATEAU_STREAK (=3)` to `ADAPTIVE_PLATEAU_STREAK (=5)`.
3. **Child deduplication:** `config_hash (sha256) + find_duplicate_config ; add_node` by default rejects any candidate whose hash collides with an existing node.
4. **Experience-library mutation priors:** `suggest_priors(comp_meta)` does keyword matching against every heading in `knowledge/experience.md` and returns evidence-annotated bullets verbatim.

D Sweep (Phase D-2..D-6, 5 competitions, first comprehensive sweep): 5/5 all wins

s3e3 (AUC, small sample of 1,677 rows), s3e7 (AUC), s3e1 (RMSE, geographic features), s3e19 (SMAPE, TimeSeriesSplit), s3e11 (RMSLE, 360k rows) — covering 4 metric families, 2 CV schemes, and 3 orders of magnitude of data scale, all surpassing the linear-iteration best within 9–21 evaluations (full numbers in the Section 2 master table, #4–8).

E Sweep (Phase E-1..E-4, 4 competitions, completing the remaining competitions + two revenge matches)

- **E-1 s3e9 revenge match:** v1 had lost with 12.07459 to linear's 12.07003 (Section 3). With the ensemble-default node space, v2 reached **12.070034** at the 14th evaluation (the BLEND seed itself) — digit-for-digit identical to the full-precision score of linear iteration's 7-way seed-bagged blend (**exact tie**), while leaving v1's single-model ceiling behind by 0.004556. The search mechanism discovered on its own the seed-bagging + weighted-blend move that linear iteration had originally found by manual trial.
- **E-2 s3e5 revenge match:** v1 had been a "statistical tie" with 0.56766 vs. linear's 0.56769. With boundary pushing (LGBBOUND, pushing the tuned-LGB's three hyperparameters that were stuck on the edges of the Optuna search box outward), v2 produced a solo-weaker (0.55784) but heterogeneous blend member that took 0.297 weight, pushing the final blend to **0.57066** — beating both v1 (+0.00300) and linear (+0.00297), a gap about 100× the v1↔linear gap (0.00003), no longer at the cut-point noise magnitude. This competition was also the first field test of the adaptive plateau mechanism, and the result was **dormant throughout** (`tie_rate` constantly 0) — because child deduplication had already removed the kind of duplicate-children spurious ties from v1; there is a dependency between the machinery: "once dedup is fixed, the trigger condition for tie-neutrality actually becomes rare."

- **E-3 s3e16 acid test:** Phase B's linear iteration had already hit the known failure mode of "rounded MAE" in this competition (raw OOF improved monotonically, while rounded OOF got worse for two rounds). With $k=800$ + coordinate-ascent weight search directly deciding against rounded MAE, v2 found **1.33563** (−0.186% relative to linear's 1.33812) — and the winning combination had raw MAE 1.35712 (worse than the linear champion's 1.35589) but better rounded MAE, a mirror-image of the Phase B failure mode, the strongest independent confirmation yet of the rule "decisions must be based on the rounded score." s3e16 is also the **only** competition in the entire v2 sweep with a real Kaggle LB anchor (Public 1.34356 / Private 1.34075, CV↔LB gap 0.00544).
- **E-4 s3e20 (structure-dominated terrain):** this competition is known to have pure structural signal (location-week historical means) that beats all GBDTs. On the single axes already manually tuned into place (W2020, WNB), the tree search honestly "tied" — proving that the manual Phase-B tuning itself had no obvious single-axis leak; the real gain came from (a) an entirely new structural axis YEARWEIGHTS (generalizing the anomalous-year down-weighting concept to 2019/2021) and (b) the JOINT lineage moving 4 structural knobs together in one step, compositely reaching the pure-structure best of 21.0589 (−0.425%). Finally, adding the marginal GBDT blend reached 21.0332 (−0.546%), but **this blend weight was fit directly against the same OOF on only 3-fold LOYO CV**, is low-confidence, and is treated as a marginal finding within CV-noise range (see the honest caveat in Section 12). This competition's evaluation cost was near zero (38 nodes + 15 backtracks in only 33.3 seconds), showing that "structure-dominated" and "model-dominated" terrains differ not only in the shape of the solution but also vastly in how scarce the search budget is.

E Sweep Wrap-up

Across the two rounds of the D+E sweeps, v2 tested a total of 9 competitions (D's 5 first-tested + E's 4: 2 that overturned/strengthened v1 results + 2 all-new acid tests), for a result of **9 wins, 0 losses** (only s3e20's blend-overlay part is marked low-confidence; the pure-structure body still wins robustly). Adding v1's 1 win, 1 tie, 1 loss, v1+v2 combined have covered 10 of the total 10 competitions (s3e9/s3e5 each have two runs, v1+v2, see the Section 2 master table).

7. Priors vs. Local Insight

`suggest_priors`'s "prior hit rate" is an opportunity to systematically quantify "how much idea injection is worth." The informed vs. uninformed win rates across the D sweep's 5 competitions: s3e3 14.3% vs. 14.3% (tie), s3e7 62.5% vs. 16.7% (prior clearly superior), s3e1 100% vs. 62.5%, s3e19 33% vs. 44.4% (prior actually slightly worse), s3e11 100% vs. 18.2%. The E sweep's 4 competitions continue the same pattern: s3e9 v2 informed 36.4% vs. uninformed 0% (small sample), s3e5 v2 33.3% vs. 0%, s3e16 38.5% vs. 0%, s3e20 informed 33.3% vs. uninformed **42.9%** (prior-annotated nodes actually had a slightly lower win rate than no-prior nodes).

A finding that recurs across all 9 D+E competitions: "priors set the floor, local insight sets the ceiling." The priors (experience library) consistently and correctly "nominate what directions to try," their value being mainly to **avoid wasting compute on known dead ends**. But in every competition,

the **single largest lever that truly opens up the score gap always comes from that competition's own EDA/comp-local insight or from a structural change to the search mechanism itself**, not from experience-library matching: s3e3's tenure-prune, s3e1's top-code clip, s3e19's auto_scale, s3e11's depth-boundary-push, and the two examples newly added in the E sweep — s3e5 v2's LGBBOUND (the boundary push itself, not a direct hit from some experience-library bullet), and s3e20's YEARWEIGHTS/JOINT (an all-new structural axis + joint move, neither of which is a rule in the experience library). Section 10's "boundary-push in 3 competitions" and "burst 3/3" are precisely two concrete, repeatable mechanized versions of this finding.

8. Honest Caveats (v1+v2, 9 competitions)

- **v1's three competitions:** s3e9's tree search structurally cannot reach the blend space (already overturned with v2); s3e5's "tie" is of a magnitude so small that a single sample's cut-point assignment could flip it; there is no knowledge transfer among the three trees.
- **D sweep's 5 competitions:** all are OOF-only, unverified on the Kaggle LB; s3e19 has the double optimistic bias of fold-5 double-dip overlaid with OOF fitting (see Section 12 for detail); s3e7 exhibited a search-level livelock during the sweep (since fixed).
- **E sweep's 4 competitions:** s3e16 is the only one with a real LB anchor; the other 8 (including v1) are still OOF-only; s3e20's GBDT-blend overlay part is a marginal finding fit directly against the same OOF on 3-fold LOYO CV, and is low-confidence.

The full, consolidated list of honest caveats (including v3) is in Section 12.

9. v3: Six Rules and the F-2 Validation Runs

Six Rules (`tree_search/harness_v3.py` , converging the manual findings from the v1→v2 sweeps into the harness's default behavior)

1. **Budget and stopping policy** (`init_budget` / `update_phase` / `should_stop`): default total budget of 60 evaluated nodes; the phase machine goes exploit → explore_burst → stopped — once all known lineages plateau, it is forced into explore_burst (the driver script is expected to inject 5–8 new long-range lineages), and after the burst starts, if 20 consecutive evaluations fail to refresh the global best it stops, with a hard stop upon reaching the total budget in any case. Evidence: E-5's 80-node curve — all 3 explore-phase improvements came from the forced burst, and without a stopping rule, 28 idle evaluations (35% of the 80-node budget) were wasted.
2. **Dedup consumes budget** (`add_node` 's dedup path): when the same parent's candidate is rejected by dedup twice in a row, a `status="failed"` placeholder child is burned, letting the parent naturally reach `MAX_CHILDREN_PER_NODE` . Evidence: while building E-5, it was found that v2's dedup rejection did not consume expansion quota, which once let the kitchen-sink blend lineage's fallback pool exhaust and then propose the same config infinitely, stalling the search at 50/80 nodes.

3. A post-plateau solo breakthrough automatically reopens the blend lineage

(`reopen_blend_lineage_on_solo_breakthrough`): when a solo node becomes the new global best, any already-plateaued blend lineage is automatically reopened. Evidence: D-6's (s3e11) whole-tree final best was a depth-12 CatBoost solo that appeared only after the phase-1 BLEND lineage had already plateaued; originally that gain required manually opening a "phase 2."

4. **Boundary pushing as a first-class mutation type** (`boundary_candidates`): automatically flags any hyperparameter falling within `edge_frac = 0.05` of the edge of its declared search space, and proposes candidates that push outward. Evidence: the single largest lever in each of D-6/E-2/E-3 (s3e11 depth 10→12, s3e5 v2's LGBBOUND, s3e16's `learning_rate`) was this pattern (see Section 10 for detail).

5. **Weight search defaults to k=800 + coordinate-ascent refinement**: `k` changes from each caller doing its own thing to a unified 800, and a round of coordinate-ascent refinement is run after the coarse search. Evidence: E-2/E-3 show a coarse grid **silently** ties (multiple candidate weight vectors round to the same discretized score), so the search must be simultaneously wide enough (k=800) and add refinement to find the true optimal basin.

6. **Metric-aware blend cost guardrail** (`eval_blend_with_cost_guard`): only if a blend evaluation exceeds a threshold (default 45 seconds) does it automatically coarsen the weight-search budget, and it **always leaves an explicit warning record**, never coarsening silently. Evidence: C-2c's (s3e5) QWK blend nodes cost ~45–46 seconds, catching up to the solo-node magnitude; E-2 further shows that silent coarsening only ties (losing true signal) without anyone noticing.

F-2 Validation Runs (s3e7, s3e14, rerun end-to-end with v3's default automatic policy)

Validation question: can harness_v3's default automatic policy (phase machine + auto-stop + dedup-consumes-budget + reopen trigger + boundary pushing + k=800 refinement + cost guardrail) run end-to-end in a real run, not regress versus v2, and does the phase machine actually do substantive work?

Answer: both, in both competitions. And in both, all the gain beyond the v2/v1 trees came 100% from the phase machine's forced explore burst.

- **s3e7:** v2 had beaten linear's 0.899893 with 0.900242 (D-3). In v3, the exploit phase exhausted all 9 first-generation lineages' plateau quotas at the 39th evaluation, reproducing v2's 4-way SEEDBAG blend (0.900054, eval 11) as the exploit ceiling; the explore burst triggered automatically at eval 39, injecting 5 long-range solos (DART/extra-trees/deep CAT/lossguide-XGB/depth2-LGB, all solo-worse) + 1 kitchen-sink mega-blend (38 members, rank-space Dirichlet + coordinate-ascent); the mega-blend took **0.900455**, beating both v2 (+0.000213) and linear (+0.000562). It stopped at the 60/60 hard cap, with 13 evaluations after the burst failing to refresh the best (< 20 patience, the numeric cap arriving first).
- **s3e14:** the v1-proto had beaten linear's 340.59891 with 340.52635 (C-2b). In v3, it overtook linear at eval 9, tied the v1-proto at eval 11 (1 evaluation later, same member set), and after grinding the exploit phase to 340.45150 (eval 16) went 22 evaluations without improvement; the explore burst triggered at eval 38, and a 34-member kitchen-sink mega-blend (of which 28 members retained >0.005 weight, unlike s3e7's 29/38 zeroed out — on this noisier target, "breadth averaging" is itself

signal) pushed the score to **340.35572**, beating both the v1-proto (0.17063) and linear (0.24319). It likewise stopped at the 60/60 hard cap, with 16 evaluations after the burst without improvement.

Boundary pushing's 4 confirmations: s3e7's `boundary_candidates()` automatically flagged that root's `max_depth` was stuck at the lower bound of the Optuna box ([3,12], 3 selected), and the automatically generated pushing mutation (depth 2) was itself solo-worse but took 0.114 weight as a burst mega-blend member — the 4th confirmation data point that "the boundary is worth pushing," and this time the harness found it **automatically**, not a human re-reading the Optuna trial table. s3e14's boundary check ran but honestly found nothing (the nearest hyperparameter was ~13% log-space distance from the box edge) — precisely the expected behavior of this mechanism: it checked, honestly found nothing, rather than being forced to dig something up.

Field results of the other rules: dedup-consumes-budget triggered 3 times in s3e7 (duplicate proposals replayed during crash-resume, all correctly burning placeholder nodes with zero wasted evaluation budget); reopen-on-breakthrough **never** triggered in either competition (blend led from very early on, no scenario of a solo catching up later); the cost guardrail **never** triggered in either competition (a 38-member AUC blend was only 11.6 seconds < the 45-second threshold).

10. The Full Ten-Competition Coverage Picture

Consolidating each competition's "best result ever reached by v2/v3" against the linear-iteration best score (see the `ten_comp_coverage` block in `docs/tree_facts.json`, taking per-competition min/max minus `linear_reference`, with the verdict determined unconditionally by metric direction):

Of 10 competitions, 9 wins, 1 exact tie, 0 losses — the only "exact tie" is s3e9 (v2's 12.070034 is digit-for-digit identical to linear iteration's full-precision 12.070034), and the other 9 (s3e14/s3e5/s3e3/s3e7/s3e1/s3e19/s3e11/s3e16/s3e20) are all clear wins. This is a qualitative leap over v1's three-competition "1 win, 1 tie, 1 loss" — the structural gap (s3e9's single-model space) was thoroughly solved by v2's ensemble-default node space, and the originally tied competition (s3e5) was turned into a clear win by v2's boundary-pushing mechanism.

This full run of 15 executions with 10-competition coverage yields three repeatable mechanistic findings:

1. **Priors set the floor, local insight sets the ceiling** (detailed in Section 7) — experience-library hits consistently avoid wasting compute on known dead ends, but the levers that truly open up the score gap (tenure-prune, top-code clip, `auto_scale`, depth-boundary-push, LGBBOUND, YEARWEIGHTS/JOINT) all come from that competition's own EDA or from a structural change to the search mechanism.
2. **explore burst + kitchen-sink mega-blend: 3/3** — in all three competitions where "the phase machine forced an exploratory burst" (E-5's s3e3 scale, F-2's s3e7, F-2's s3e14), all the post-exploit-phase gain came from the kitchen-sink mega-blend injected by the burst itself, never from a single hand-written long-range solo lineage contributing alone. The three competitions' gains were,

respectively, +0.001527 (s3e3 scale, against the exploit ceiling 0.843524), +0.000401 (s3e7, against 0.900054), and s3e14 dropping from 340.45150 to 340.35572 (improvement 0.09579).

3. **boundary-push: 3 competitions** — s3e11 (D-6, CatBoost max_depth 10→12, solo 0.295779→0.295461), s3e5 v2 (E-2, LGBBOUND pushing max_depth from 3 to 2, solo score 0.55784 but taking 0.297 blend weight), s3e16 (E-3, learning_rate pushed from the Optuna box edge 0.0102 to 0.005, solo rounded MAE 1.33979→1.33950) — the single largest lever in all three competitions stems from the pattern "the Optuna optimum is stuck on the search-space boundary." s3e7's F-2 run additionally provides a 4th confirmation data point (the boundary-push member took 0.114 blend weight), the difference being that this time v3's `boundary_candidates()` found it automatically rather than a human re-reading the Optuna trial table — the cleanest evidence for the v3 design principle of "turning human intuition into a system default."

11. Scaling-Curve Chapter (Phase E-5, `docs/scaling_experiment.md`)

D-2's 22-node s3e3 sweep found AUC 0.841442, but there was still visible headroom at the time (four lineages had plateaued, but the search hit the 22-node cap before exhausting the candidate space). E-5 extends the same search regime to a budget of 80 evaluated nodes and directly measures the score-vs-evaluations curve; the questions are: where does the curve flatten, and are the late-stage backtracks worth it?

Curve trajectory (verbatim from the `curve` field of `experiments_tree_scale.json`, mechanically extracted by `docs/scripts/build_tree_facts.py`'s `curve_summary()`, cross-checked against the same numbers manually compiled in `docs/scaling_experiment.md`): 10 global-best refreshes over the whole run, of which the exploit phase (eval 1–39) contributed 7 and the explore phase (from eval 41) contributed 3. The last improvement in the exploit phase was at eval 39 (0.843524), after which the exploit phase itself improved no further; after the explore burst triggered, all 3 improvements concentrated in eval 45–52 (the KITCHENBLEND lineage), and then, until eval 80 (the end of the run), there were **no further improvements at all** — the idle tail was as long as 28 evaluations, 35% of the 80-node budget. The whole run's actual wall-clock time (including cache reuse and failed placeholder nodes) was 97.4 seconds.

Stage-4 budget rules (original text from `docs/scaling_experiment.md`, confirmed via the `v3_constants` block in `docs/tree_facts.json` to have been faithfully encoded into `harness_v3.py`'s defaults): (1) default node budget 60 (exploit ~35–40 + a forced explore burst of 5–8 long-range lineages); (2) stop after 15–20 consecutive evaluations without improvement post-burst (v3 default 20); (3) numeric cap 60, a hard stop regardless of whether the phase machine triggers. These three rules are the direct source of `harness_v3.py`'s `DEFAULT_TOTAL_BUDGET=60`, `EXPLORE_BURST_MIN/MAX=5/8`, and `DEFAULT_POST_BURST_PATIENCE=20` constants.

Cross-calibration with all 15 runs (the `budget_efficiency` block in `docs/tree_facts.json`, mechanically recomputing per run the "at which evaluation the best solution appeared / total evaluations" ratio and the tail idle-evaluation count): the best/total ratio across 15 runs ranges from

0.10 (s3e9 v1) to 1.00 (s3e20, whose structure-dominated terrain has near-zero evaluation cost and where the very last node refreshed the best), with a mean of about 0.65; the idle tail ranges from 0 (s3e20) to 28 (s3e5 v1's 40-node run, and s3e3 scale's 80-node run — the same 28-evaluation idle tail recurring at $2\times$ the budget scale, showing this is a characteristic of the harness's own search dynamics, not merely a coincidence of a specific budget size).

12. Honest Caveats (v1+v2+v3, all 15 runs)

1. **Of all 15 runs, only s3e16 (E-3) has a real Kaggle LB anchor** (Public 1.34356 / Private 1.34075, CV↔LB gap 0.00544) — the other 14 runs' conclusions that "the tree best beat the reference" hold only on **OOF** scores, unverified on the Public/Private Leaderboard. It was an ironclad rule for this weekend batch run to "not touch any token"; an OOF win does not guarantee an LB win, especially when the weight search/post-processing parameters were themselves fit against the same OOF.
2. **s3e19's fold-5 double-dip + OOF-fitting caveat**: linear iteration's original 10.01946 already carries the double-use optimistic bias of "fold 5 being simultaneously the Optuna tuning target and one of the 5 OOF folds"; the tree search's 9.75707 further layers on top of this the two levels of `auto_scale` ($\times 1.02$ scalar) and seed selection, both fit directly against the 114,000-row OOF. The honest reading is "the true SMAPE should be significantly below 10.02, but should not be read directly as 9.76."
3. **s3e20's GBDT-blend overlay part is low-confidence**: the pure-structure node (21.0589) is a robust conclusion, but the final 21.0332 with GBDT overlaid fits a tiny 2.17% weight directly against the same OOF on only **3-fold LOYO CV**, and is very likely CV noise rather than true signal, consistent with Phase-B's existing residual-diagnostic conclusion (sensor features contain no predictable signal for this target beyond the structure).
4. **v3's auto-stop (patience counter) never truly triggered in either F-2 field run** — in both s3e7 and s3e14, the explore burst kept improving the global best, the patience counter kept getting reset, and ultimately the 60-node numeric cap (not the patience rule itself) ended the search. The patience path itself was only validated end-to-end once, in a small-budget driver-script smoke test (`stop_reason` correctly outputting "2 evals without improvement post-burst"); **in a real run it will first be truly tested on a competition where the burst is a dud, and there is no such data point yet** — this is the one part of the claim "auto-stop works" currently unsupported by field evidence, recorded honestly rather than pretending it is validated.
5. **Driver-script-level bugs exposed by the F-2 validation runs (not harness_v3.py itself)**: s3e7 required 3 restarts — (a) the blend-vs-solo dispatch logic wrongly judged the type by the literal lineage name "BLEND" rather than the node's own `kind`; (b) module-level driver state (lineage names, burst flags, `node_results`) did not survive resume, fixed by moving this state into `search_state` itself; (c) one CatBoost training (EXPL_CATDEEP) had its native `fit()` hang for 28 minutes, and `signal.alarm` cannot interrupt a native call that never returns to Python bytecode, so it could only be manually killed and rerun (the rerun itself took only 54.8 seconds). s3e14, meanwhile, hit the process's own 35-minute wall-clock limit at 59/60 nodes, and additionally 6 DART evaluations (107–140 seconds each, with OOF MAE as high as 6144–6544, about $18\times$ the baseline) wasted about 12 minutes for nothing — `harness_v3.py` itself was **unchanged** in both

validation runs, and all three bugs are driver-script-level lessons, see the three pre-deployment engineering requirements in Section 13.

13. Stage 4 Formal Recommendations

v3 as the Default Loop

`harness_v3.py` (phase machine / dedup-consumes-budget / blend reopen / boundary pushing / $k=800$ + refinement weight search / cost guardrail) should be the **default loop** for Stage 4 tree search, because: (1) both F-2 validation runs ran end-to-end and both refreshed the all-time best for their competition (s3e7 0.900455, s3e14 340.35572), with `harness_v3.py` itself unchanged during validation; (2) the explore-burst mechanism contributed the sole post-exploit-phase gain in all three triggers, 3/3 (Section 10); (3) boundary pushing was confirmed as one of the single largest levers in four independent competitions.

Three Engineering Requirements That Must Be Completed Before Deployment (gaps directly identified by the F-2 honest caveats, not a forged new wish list)

1. **Resume state contract:** the driver script's own runtime state (lineage-name mapping, burst-injected flag, `node_results` scratch) must be explicitly defined as part of `tree["search_state"]` and persisted along with the tree, rather than left in module-level variables of the driver script — 1 of the 3 restarts in s3e7's F-2 validation run stemmed directly from this; the fix was made in that run's driver script but has not yet been distilled into a formal contract for the harness or the driver-script template.
2. **Subprocess-level evaluation timeout:** `signal.alarm` cannot interrupt a native `fit()` that never yields control back to Python bytecode (in s3e7's F-2 one CatBoost training hung for 28 minutes). Each node's solo/blend evaluation should run in a separate subprocess (`subprocess / multiprocessing`), with the parent process forcibly killing a timed-out child, rather than relying on an in-process signal.
3. **Sanity gate for burst seeds:** the 6 DART long-range solo evaluations in s3e14's F-2 produced OOF MAE as high as $18\times$ the baseline and each took 107–140 seconds ($2\text{--}3\times$ a typical node) — the long-range seeds injected by the burst need a "before/during each seed evaluation" wall-clock and preliminary-score sanity check (e.g., whether the training loss is diverging, whether the first few rounds' wall far exceeds the median of historically comparable nodes), aborting an obviously runaway long-range attempt early rather than letting it run to completion only to discover it was a waste.

Budget Rules (validated in Phase E-5, encoded as `harness_v3.py` constants, Section 11)

- Default total node budget **60** (exploit phase $\sim 35\text{--}40$ + a forced explore burst of 5–8 long-range lineages).

- **The explore burst must not be omitted**, even if the exploit phase "looks converged" — three independent pieces of evidence (E-5/F-2×2) show the burst is the sole source of post-exploit gain.
- Stop after **15–20** consecutive evaluations without improvement post-burst (v3 default 20); if neither the phase machine nor the numeric signal triggers cleanly, fall back to the **60**-node numeric cap (E-5 validation: the mean best/total ratio across all 15 runs is 0.647, ranging 0.1–1.0, with E-5 itself at 0.65 — the cap leaves ample headroom and will almost never truncate a real improvement).

14. Reproduction Commands (v1 + v2 + v3 + scaling experiment, 15 runs)

```
cd /home/tjyen/ai_agents/kaggle

# --- v1 (harness.py, Phase C-2a/b/c) ---
uv run python3 tree_search/run_s3e9.py      # single-model node space
uv run python3 tree_search/run_s3e14.py     # solo + blend node space
uv run python3 tree_search/run_s3e5.py      # discretized-metric (QWK-after-rounder) generalization test

# --- v2 D sweep (harness_v2.py, Phase D-2..D-6) ---
uv run python3 tree_search/run_s3e3.py      # AUC, small sample (1,677 rows)
uv run python3 tree_search/run_s3e7.py      # AUC, 42k rows
uv run python3 tree_search/run_s3e1.py      # RMSE, geographic features, 37k rows
uv run python3 tree_search/run_s3e19.py     # SMAPE, TimeSeriesSplit
uv run python3 tree_search/run_s3e11.py     # RMSLE, 360k rows

# --- v2 E sweep (harness_v2.py, Phase E-1..E-4) ---
uv run python3 tree_search/run_s3e9_v2.py   # revenge match: can ensemble-default overturn the v1 loss
uv run python3 tree_search/run_s3e5_v2.py   # revenge match: can v2 break the v1 tie
uv run python3 tree_search/run_s3e16_v2.py  # acid test: rounded MAE
uv run python3 tree_search/run_s3e20_v2.py  # structure-dominated terrain

# --- v2 E-5 scaling experiment ---
uv run python3 tree_search/run_s3e3_scale.py # 80-evaluation-node budget curve

# --- v3 F-2 validation runs (harness_v3.py) ---
uv run python3 tree_search/run_s3e7_v3.py   # interruptible/resumable
uv run python3 tree_search/run_s3e14_v3.py  # interruptible/resumable
```

All 15 run scripts are interruptible/resumable (each node, once written, is atomically written to its corresponding `experiments_tree*.json` via temp-file + `os.replace`). To rerun from scratch, first delete the corresponding tree-state file and `tree_search/cache_<comp>*/` (both regenerable, already-gitignored scratch).

Fact extraction and validation:

```
uv run python3 docs/scripts/build_tree_facts.py
uv run python3 .claude/skills/kaggle-report/assets/verify_report.py docs/tree_search_prototype.md docs/tr
bash .claude/skills/kaggle-report/assets/md2pdf.sh docs/tree_search_prototype.md
```